

Laporan M2 Tugas Besar 1

IF2010 Pemrograman Berorientasi Objek 2025/2026

Nimonspoli

Kode Kelompok : D3R

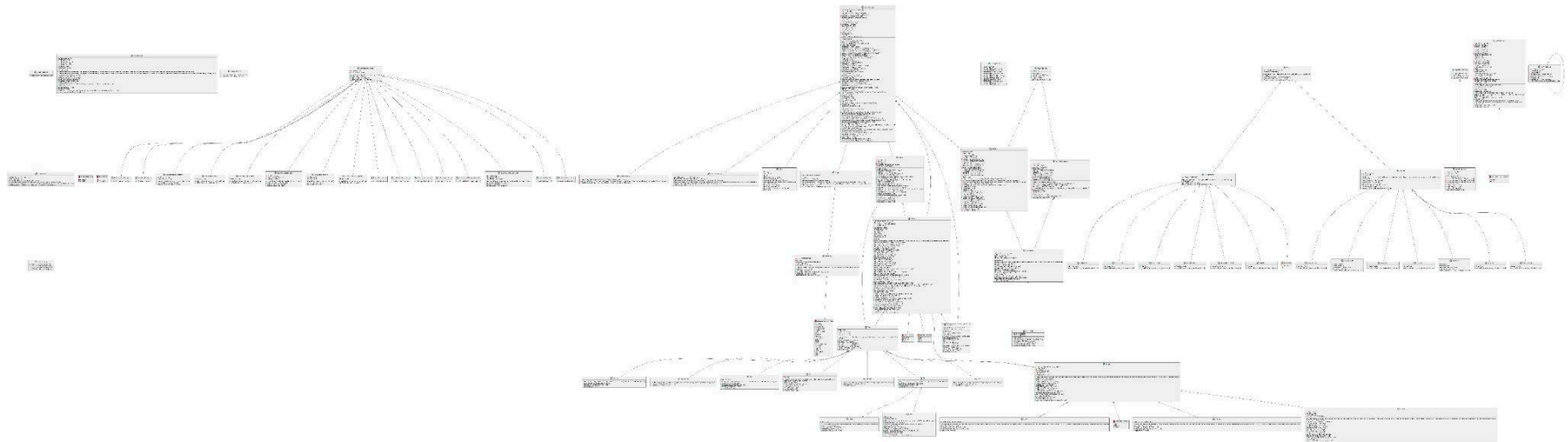
Nama Kelompok : Completing a Tubes that doesn't exist

1. 13524003 / Faiq Azzam Nafidz
2. 13524013 / Anindya Naufal Pinasthika
3. 13524048 / Josh Reinhart Zidik
4. 13524050 / Raysha Erviandika Putra
5. 13524084 / Dzaki Ahmad Alhusainy



D3R - Completing a Tubes that doesn't exist

1. Diagram Kelas



Representasi diagram kelas secara menyeluruh dapat diakses melalui file berikut [PDF nimonspoli.pdf](#)

2. Deskripsi Kelas

2.1. Kelas GameManager

Kelas ini menjadi representasi dari sistem yang menjalankan keseluruhan permainan sehingga memiliki relasi yang berhubungan dengan semua kelas yang tersedia pada program ini.

- turn** dengan tipe `int` yang menunjukkan giliran permainan saat ini,
- maxTurn** dengan tipe `int` yang menunjukkan batas maksimum jumlah giliran dalam permainan,
- activePlayerCount** dengan tipe `int` yang menunjukkan jumlah pemain yang masih dapat bermain,
- playerCount** dengan tipe `int` yang menunjukkan jumlah pemain yang berpartisipasi,

- e. **initialCurrency** dengan tipe `int` untuk menyimpan data jumlah uang awal yang diterima setiap pemain,
- f. **players** dengan tipe `vector<Player*>` yang menyimpan daftar seluruh pemain dalam permainan,
- g. **board** dengan tipe kelas `Board` yang merepresentasikan papan permainan beserta susunan petaknya,
- h. **commandHandler** dengan tipe kelas `CommandHandler` untuk memproses perintah dari setiap pemain dalam permainan,
- i. **deckSkill** dengan tipe kelas generic `Static CardDeck<T>` yang berisi kumpulan kartu skill yang digunakan selama permainan,
- j. **dechChance** dengan tipe kelas generic `Static CardDeck<T>` yang berisi kumpulan kartu kesempatan yang digunakan selama permainan
- k. **deckCurrency** dengan tipe kelas generic `Static CardDeck<T>` yang berisi kumpulan kartu dana umum yang digunakan selama permainan
- l. **Logger** dengan tipe kelas `Logger` yang mencatat riwayat keadaan permainan pada setiap langkah,
- m. **currentTurnPlayer** dengan tipe `Player*` yang menunjukkan pemain yang sedang mendapatkan giliran,
- n. **dice** dengan tipe kelas `Dice` yang merepresentasikan nilai dadu yang digunakan dalam permainan.

Methods:

- a. `isGameValid()` melakukan validasi kondisi data setelah inisialisasi struktur data dari config
- b. `runGame()` melakukan inisialisasi dari permainan secara keseluruhan
- c. `auction(Tile*)` menjalankan event auction tile property secara global ke semua player
- d. `initBoard()` populasi board baru dengan data tile, termasuk tile properti dari state properti di config
- e. `initPlayers()` populasi player dengan data player dari config
- f. `initStateLogs()` populasi logger dengan data statelog dari config
- g. `initSkillDeck()` populasi deck kartu kemampuan dari config
- h. `rollDice()` melempar dua dadu secara acak dan menerapkan hasilnya pada giliran pemain saat ini.
- i. `rollDice(int, int)` menggunakan nilai dadu yang ditentukan (untuk perintah ATUR_DADU)
- j. `initAutoUseDecks()` menginisialisasi deck atau kumpulan kartu yang berasal dari kelas `AutoUseCard`
- k. `drawSkillCard(Player*)` mengambil kartu skill untuk pemain tertentu dan memberikannya ke pemain tersebut.
- l. `nextTurn()` memindahkan giliran ke pemain berikutnya serta memperbarui status permainan terkait.
- m. `pay(Player*, int, Player*)` memproses pembayaran sejumlah uang dari pemain (debtor) kepada pihak penerima (creditor).

- n. `sellPropertyToBank(Player*, Property*)` menjual properti milik pemain ke bank untuk mendapatkan uang tunai.
- o. `handleBankruptcy(Player*, int, Player*)` menangani kondisi ketika pemain tidak mampu membayar lagi dan dinyatakan bangkrut.
- p. `assetAcquisition(Player*, Player*)` memindahkan aset milik pemain yang bangkrut kepada pemain lawan yang mengalahkannya.

2.2. Kelas Player

Kelas ini berfungsi sebagai representasi dari setiap pemain yang terdapat dalam permainan, setiap objek dari kelas ini menggambarkan seorang pemain yang unik. Kelas Player memiliki atribut utama yaitu:

- a. **username** dengan tipe `std::string` sebagai *identifier* unik dari setiap player,
- b. **currency** dengan tipe `int` yang menunjukkan jumlah uang yang dimiliki oleh pemain,
- c. **currentStatus** dengan tipe `enum list PLAYER_STATUS` yang menunjukkan status pemain saat ini, `PLAYER_STATUS` memiliki 3 status berbeda yaitu `ACTIVE`, `BANKRUPT`, dan `JAILED`.
- d. **deck** dengan tipe kelas generic `CardDeck<T>` yang menunjukkan kartu kemampuan yang dimiliki pemain pada round tertentu.
- e. **currentTile** dengan tipe kelas `Tile` yang menunjukkan petak lokasi pemain.
- f. **discountValue** dengan tipe `float` untuk menyatakan diskon yang dimiliki player
- g. **jailTurnCount** dengan tipe `int` menyatakan jumlah turn player berada di dalam jail
- h. **activeCardEffect** dengan tipe `enum list CARD_EFFECT: NULL, DISCOUNT, SHIELD` menunjukkan status kartu aktif pada pemain saat ini
- i. **effectTurns** dengan tipe `int` untuk menunjukkan jumlah giliran efek kartu masih berjalan pada pemain
- j. **doubleCount** dengan tipe `int` untuk menyimpan data jumlah dadu *double* yang diterima pemain pada suatu *turn*,
- k. **canUseCard** bertipe `bool` untuk menyatakan apakah player bisa menggunakan kartu skill

Methods:

- a. `buyBackMortgaged(Tile*)` fungsi untuk membeli kembali properti milik player yang telah digadaikan
- b. `operator--(int)` fungsi untuk menambahkan currency
- c. `operator+=(int)` fungsi untuk mengurangi currency

- d. `activateDiscount(float, int)` mengaktifkan efek *discount* pada pemain sebesar `float` persen dan selama `int` giliran
- e. `activateShield(int)` mengaktifkan efek *shield* pada pemain selama `int` giliran
- f. `resetCardUse()` mengubah atribut **canUseCard** untuk mengizinkan pemain menggunakan kartu kembali
- g. `endTurnEffects()` melakukan *update* kepada atribut **effectTurns** pada setiap akhir turn
- h. `pay(int, Player*)` melakukan proses pembayaran dari pemain ke lawan sejumlah `int` disertai pengecekan efek kartu dan `try catch` untuk *exception* yang dapat terjadi
- i. `addSkillCard(SkillCard* card)` menambah **skillCard** pada deck milik pemain, metode ini selalu dipanggil pada awal turn
- j. `removeSkillCard(int index)` menghapus **skillCard** dari deck pemain
- k. `moveTo(Tile&, bool)` fungsi untuk berpindah tile dengan mengubah atribut `currentTile`, serta boolean check apakah melewati tile GO
- l. `mortgageProperty(Tile*)` fungsi untuk menggadai properti milik player
- m. `setToJailed()` untuk mengubah status pemain ke `JAILED`
- n. `getTotalWealth(const Board* board)` untuk menghitung kekayaan total pemain, termasuk aset dan properti yang dimiliki
- o. `getMaxLiquidatableValue(const Board* board)` untuk menghitung properti pemain yang bisa diuangkan
- p. `getOwnedProperties()` untuk mengembalikan seluruh **Tile** properti milik pemain

2.3. Kelas Logger

Kelas logger berfungsi untuk menyimpan log yang dimiliki game, kelas ini memiliki atribut sebagai berikut.

- a. **logs** dengan tipe `vector<StateLog>` menyimpan semua entri `StateLog`

logger memiliki satu metode `Logger::log(int, string, StateLog::ACTION_TYPE, string)` untuk menambahkan entri **StateLog** pada `int` turn, `string` username, `ACTION_TYPE` jenis aksi, dan `string` detail. Selain itu ada metode `Logger::printLog(int)` yang akan melakukan print semua log yang tersimpan sebanyak `int`.

2.4. Kelas StateLog

Kelas `StateLog` adalah kelas merepresentasikan satu entri game log. Setiap aksi signifikan dalam permainan menghasilkan satu objek `StateLog` yang ditambahkan ke vector `stateLogs` di `GameManager`. Kelas ini bersifat immutable setelah konstruksi.

- a. **turn** dengan tipe `int` menyatakan nomor giliran saat aksi terjadi.

- b. **username** dengan tipe `string` menyatakan username pemain yang melakukan aksi
- c. **action** enum `ACTION_TYPE: BUY, DICE, USECARD, GETCARD, PAYRENT, PAYTAX`
- d. **detail** dengan tipe `string` deskripsi teks dari aksi yang terjadi.

Kelas ini hanya akan berlaku untuk menyimpan data gamelog.

2.5. Kelas Board

Kelas Board adalah representasi papan permainan yang menyimpan susunan seluruh petak (Tile) dalam urutan yang benar. Board bertanggung jawab atas navigasi antar petak, mencari petak berdasarkan indeks atau kode, menghitung petak tujuan setelah pergerakan, dan menyimpan atribut global tertentu dari petak-petak yang ada. Atribut utama dari board adalah:

- a. **tiles** dengan tipe `private static std::vector<Tile*>` menyimpan semua pointer objek tile yang ada pada board.
- b. **tileCount** dengan tipe `int` menyatakan jumlah tile yang ada pada board.
- c. **goIndex** dengan tipe `int` menyatakan index eksplisit dari tile GO pada vector tiles
- d. **jailIndex** dengan tipe `int` menyatakan index eksplisit dari tile Jail pada vector tiles

Kelas ini memiliki fungsi untuk melakukan kalkulasi pergerakan player di board. Fungsi `Tile* getTile(int)` berperan dalam mengembalikan Tile tujuan pada index tertentu. Penggunaan index akan bersifat sirkular untuk merepresentasikan board yang sirkular. (`index % tileCount`). Untuk beberapa *occurence* tertentu player dapat langsung masuk ke jail, maka dari itu ada Fungsi `Tile* getJailTile()` yang mengembalikan Tile jail secara langsung. `goIndex` pada kelas ini digunakan sebagai checker apabila player telah melewati tile GO pada satu putaran dan mendapatkan gaji. Penggunaan fungsi `Tile* goToTile(Tile&, int)` dapat dipanggil untuk melakukan kalkulasi pergerakan player pada suatu turn, dimana `Tile&` adalah reference dari Tile yang sedang ditempati player, dan argumen `int` sebagai angka *dice roll* yang diperoleh. Diluar itu, terdapat fungsi helper lain yaitu:

1. `getColorGroup(string)`, untuk mendapatkan seluruh **Tile** dengan color group `string`
2. `canUpgradeProperty(Player, Tile*)`, untuk mengecek apabila sebuah **Tile** dapat di-upgrade
3. `getRailroadLevel(Tile*)`, untuk mendapatkan level dari **Tile** Railroad saat ini
4. `getUtilityLevel(Tile*)`, untuk mendapatkan level dari **Tile** Utility saat ini
5. `getNextPlayer(int currentIndex)`, untuk mendapatkan pemain terdekat di depan pemain saat ini (ditandai oleh `int currentIndex`)

6. `stringToIndex(std::string)`, untuk mengubah `string` kode menjadi `int` index
7. `goToTile(std::string, int)`, untuk mendapatkan **Tile** hasil setelah bergerak sejumlah `int` petak dari petak berkode `std::string`
8. `addTile(Tile*)` untuk menambahkan tile baru ke board.
9. `advanceFestivalEffects(Player* owner)` untuk meng-*update* kondisi festival saat ini
10. `cetakAkta(std::string code)` untuk mencetak akta dari properti saat ini

2.6. Kelas Tile

Kelas Tile merupakan kelas abstrak yang menjadi representasi setiap petak dalam permainan. Kelas ini memiliki *inheritance* untuk dua kelas lain, yaitu kelas **Property**, **cardTile**, **Festival**, **Tax**, **Go**, **Trap**, **Prison**, dan **FreeParking** yang menggambarkan petak properti dan petak aksi. Atribut yang terdapat pada kelas ini yaitu **code** dan **color** dengan tipe `string` sebagai identifier dasar untuk setiap petak permainan. Sebagai kelas abstrak, kelas ini memiliki metode *pure virtual* yaitu `runTile(Player)` untuk menjalankan aksi spesifik petak terhadap pemain yang berada di atasnya.

2.6.1. Property

Kelas Property merepresentasikan petak dalam papan yang bertipe properti. Ciri khas dari petak ini adalah bisa dimiliki oleh seorang *player* atau *bank*. Ketika *player* mendarat di petak ini, *player* memiliki kesempatan untuk mengambil alih petak properti menjadi miliknya (asalkan belum dimiliki oleh *player* lain) atau membayar sewa kepada *player* pemilik properti. Atribut yang dimiliki kelas Property:

- a. **landCost** dengan tipe `int` sebagai harga beli tanah properti,
- b. **mortgageValue** dengan tipe `int` sebagai nilai gadai properti,
- c. **festivalMultiplier** dengan tipe `int` sebagai nilai *multiplier* untuk sistem Festival, atribut ini hanya dapat bernilai salah satu di antara 1, 2, 4, atau 8,
- d. **festivalDuration** dengan tipe `int` sebagai nilai durasi yang tersisa dari sistem Festival pada petak, atribut ini dapat bernilai 1 sampai 3,
- e. **owner** dengan tipe `Player*` yang menyimpan pointer ke objek pemain yang memiliki properti tersebut dan `nullptr` jika dimiliki bank, dan

- f. **propertyStatus** dengan tipe `enum list PROPERTY_STATUS` yang menunjukkan status dari properti saat ini, `PROPERTY_STATUS` memiliki 3 status berbeda yaitu `BANK`, `OWNED`, dan `MORTGAGED`.

Kelas ini memiliki metode `runTile(Player)` yang meng-*override* metode *virtual* milik kelas `Tile` untuk menjalankan aksi spesifik pada petak properti. Selain itu, kelas ini memiliki metode *pure virtual* sebagai kelas abstrak yaitu `int getRentCost()` untuk mendapatkan biaya sewa petak, metode ini bersifat *pure virtual* karena dapat berubah tergantung pada tipe propertinya.

Petak properti dibagi menjadi tiga jenis, yaitu **Streets** (lahan), **Railroad** (stasiun), dan **Utility** (utilitas). Setiap jenis petak properti tersebut dibuat kelas dengan nama sesuai jenisnya yang mewarisi kelas `Property`.

2.6.1.1. Streets

Kelas `Streets` merupakan kelas *inheritence* dari kelas `Property`, di mana sebagai representasi jenis `Property` umum dalam permainan. Setiap `Street` termasuk dalam sebuah *color group*, yaitu kelompok properti yang berbagi warna yang sama. Dalam kelas `Streets`, atribut yang dimiliki adalah sebagai berikut:

- houseCost** bertipe `int` sebagai representasi biaya untuk membangun sebuah rumah pada `street` tersebut,
- hotelCost** bertipe `int` sebagai representasi biaya untuk membangun sebuah rumah pada `street` tersebut,
- rentCost** bertipe `vector<int> (5)` untuk menunjukkan biaya sewa saat ini,
- currentLevel** bertipe `int` untuk menunjukkan tingkat level rumah/hotel saat ini dalam sebuah `street`.

Kelas `Streets` juga memiliki *method* `int getRentCost()`. Di mana *method* tersebut meng-*override* *method virtual* `int getRentCost()` milik kelas **Property** untuk menyesuaikan biaya sewa bagi class `Streets`. Selain itu, terdapat beberapa metode spesifik kelas **Street** yaitu

- `canBuild(Player&)`, memberikan `boolean` apakah pemain `Player&` dapat membangun sesuatu di **Tile**
- `build(Player& player)` melakukan pembangunan pada **Tile**
- `runTile(Player*)` melakukan proses pembelian petak atau pembayaran sewa
- `getAssetValue()` mengembalikan nilai aset dari petak saat ini

2.6.1.2. Railroad

Kelas Railroad merupakan kelas *inheritance* dari kelas Property, di mana sebagai representasi jenis Property unik. Hal tersebut karena pemain akan langsung menjadi hak pemilik petak railroad tanpa perlu membayar suatu biaya, asalkan pemain tersebut tiba pertama pada petak railroad tersebut. Atribut kelas Railroad adalah sebagai berikut:

- a. **rentCost** bertipe `vector<int>` (4) untuk menunjukkan biaya sewa bagi petak Railroad

Kelas Railroads juga memiliki *method* `int getRentCost()`. Di mana *method* tersebut meng-*override* *method virtual* `int getRentCost()` milik kelas Property untuk menyesuaikan biaya sewa bagi class Railroad.

2.6.1.3. Utility

Kelas Utility merupakan kelas *inheritance* dari kelas Property, di mana sebagai representasi jenis Property unik. Selain memiliki sistem kepemilikan dengan petak Railroad, petak Utility memiliki sistem perhitungan biaya sewa yang berbeda, di mana harga ditentukan oleh total angka dadu yang didapatkan pemain ketika mendarat, dikali dengan faktor pengali yang bergantung pada jumlah petak *Utility* yang dimiliki oleh pemiliknya. Selain itu, makin banyak Utility yang dimiliki maka faktor pengali akan semakin besar. Kelas Utility memiliki atribut sebagai berikut:

- a. **costMultiplier** bertipe `vector<int>` (5) untuk menunjukkan biaya sewa bagi petak Railroad

Kelas Utility juga memiliki *method* `int getRentCost()`. Di mana *method* tersebut meng-*override* *method virtual* `int getRentCost()` milik kelas Property untuk menyesuaikan biaya sewa bagi class Utility.

2.6.2. CardTile

Kelas CardTile merupakan kelas *inheritance* dari kelas Tile. Sebagai representasi petak di mana pemain harus mengambil sebuah kartu ketika menginjak petak ini, Player akan mendapatkan sebuah kartu dan wajib menggunakan kartu tersebut sesaat sudah mengambil kartu tersebut.

Kelas CardTile tidak mempunyai atribut khusus, tetapi memiliki *method* `runTile(Player) override` untuk menjalankan petak Card, serta *method* `Card getRandomCard()` Di mana *method* tersebut me-*return* sebuah Card di mana pemain harus menggunakan Card tersebut.

2.6.3. Festival

Kelas Festival merupakan kelas *inheritence* dari kelas Tile. Petak festival merepresentasikan petak di mana pemain dapat memilih sebuah petak properti yang dimiliki agar dilipatgandakan saat pemain menginjak petak ini, dalam catatan harga properti yang dipilih yang dilipatgandakan akan berlangsung selama 3 giliran. Pemain juga memiliki batas 3 kali untuk melakukan penglipat gandaan sebuah properti yang dimilikinya.

Kelas Festival tidak mempunyai atribut khusus, tetapi memiliki *method* `runTile(Player) override` untuk menjalankan petak Festival, serta *method* `void addMultiplier(Tile)` Di mana *method* tersebut akan mengubah harga sewa properti milik pemain agar dilipatgandakan dari harga semulanya.

2.6.4. Tax

Kelas Tax merupakan kelas *inheritence* dari kelas Tile. Petak tax merepresentasikan petak di mana pemain harus membayar Pajak Penghasilan (PPH), di mana pemain harus dibayar dalam bentuk sejumlah *flat*/presentase total kekayaan, atau Pajak Barang Mewah (PBM), di mana pemain harus membayar nilai yang didefinisikan dalam file konfigurasi langsung/

Kelas Tax tidak mempunyai atribut khusus, tetapi memiliki *method* `virtual runTile(Player) override` untuk menjalankan petak tax di mana pemain harus membayar sejumlah pajak berdasarkan aturan yang disebutkan sebelumnya. Selain itu, kelas tax juga mempunyai *method* `virtual void payTax(Player*) = 0;` sebagai *method* pure abstract yang akan diturunkan dan dipakai oleh anak class PPH dan PBM.

2.6.4.1. PPH

Kelas Tax PPH merupakan kelas *inheritence* dari kelas Tax. Fungsi dari anak kelas PPH akan menghitung total pajak pada petak Tax PPH berdasarkan pilihan player, yakni pembayaran flat atau pembayaran presentase total kekayaan. Kelas PPH tidak mempunyai atribut sebagai berikut:

- i. **flatTax** bertipe `int` sebagai representasi biaya pajak PPH dalam pembayaran flat.
- ii. **taxPrecentage** bertipe `int` sebagai representasi biaya pajak PPH dalam pembayaran presentase dari total kekayaan pemain.
- iii. **PPH_OPTION** enum `FLAT`, `PERCENTAGE` sebagai representasi pilihan PPH yang akan dilakukan, yakni membayar secara flat/membayar secara presentase.

Kelas PPH memiliki *method* `void payTax(Player*)override` untuk menjadi gateway, `void payPphTax(Player*, PPH_OPTION)` untuk menjadi pilihan antara pembayaran PPH akan ditentukan menjadi flat atau presentase, `int calculateFlatTax() const` untuk mengkalkulasikan jumlah flat tax yang akan diberikan, `int calculatePrecentageTax(const Player&) const` untuk mengkalkulasikan jumlah presentase tax yang akan diberikan.

2.6.4.2. PBM

Kelas Tax PBM merupakan kelas *inheritence* dari kelas Tax. Fungsi dari anak kelas PBM akan menghitung total pajak pada petak Tax PBM, di mana pajak didefinisikan pada file konfigurasi yang sudah dibuat. Kelas PPH tidak mempunyai atribut berupa:

- i. **fixedTax** bertipe `int` sebagai representasi biaya pajak PBM di mana nilai tersebut didapat dari konfigurasi.

Kelas PBM juga memiliki *method* `void payTax(Player*)override` untuk menjadi alur logika pembyaran pajak secara PBM.

2.6.5. Go

Kelas Go merepresentasikan petak Mulai sebagai petak awal permainan. Petak ini menjadi sumber pendapatan bagi para pemain dengan memberikan uang kepada pemain dari bank ketika pemain melewati atau berhenti di petak ini (kecuali kondisi khusus seperti sedang dipindahkan ke penjara). Kelas go memiliki atribut:

- a. **payment** bertipe `int` sebagai representasi biaya currecy yang akan diberikan ke Player tiap kali Player melewati petak go.

Perlakuan kelas Go direalisasikan melalui *method* `givePayments()`. Karena perlakuan petak saat pemain mendarat di petak ini sudah direpresentasikan oleh `givePayments()`, isi `runTile()` tidak melakukan apapun.

2.6.6. Prison

Kelas Prison merepresentasikan petak Penjara yang ada dalam papan permainan. Petak penjara sendiri memiliki dua bagian di dalamnya, yaitu hanya mampir (pemain dianggap sebagai pengunjung saat mendarat) dan di dalam penjara (pemain berstatus dipenjara). Karena ketika pemain ketika mendarat dianggap sebagai pengunjung saja, implementasi dari *method* `runTile()` pada kelas ini tidak memberikan efek apapun terhadap pemain. Kelas ini memiliki atribut:

- a. **fee** bertipe `int` sebagai representasi biaya agar Player bisa bebas dari penjara

kelas ini memiliki *method* tambahan yaitu `checkJailed()` untuk mengecek apakah suatu player di dalam sebuah penjara atau tidak, `payFee()` untuk mengatur sistem pembayaran keluar dari penjara, `setJailed()` (memberikan status dipenjara kepada pemain) dan `freeFromJailed()` (menghilangkan status dipenjara kepada pemain). Terdapat `runTile(Player*) override` dipanggil memanggil `payFee()` dan `freeFromJailed()` jika `payFee()` berhasil.

2.6.7. Trap

Kelas Trap merepresentasikan petak Pergi ke Penjara pada papan permainan. Pemain yang mendarat di petak ini akan dipenjara, yaitu posisi pemain dipindahkan ke petak Penjara dan diberikan status tahanan. Kelas ini meng-*override* fungsi `runTile()` yang berfungsi untuk memenjarakan pemain sesuai dengan penjelasan sebelumnya.

2.6.8. Free Parking

Kelas ini hanya mengimplemtasi tile kosong dan berfungsi sebagai *rest area*. Jika pemain berhenti di sini, tidak ada aksi apa pun yang terjadi.

2.7. Kelas Generik CardDeck

Kelas generik ini adalah abstraksi dari penggunaan banyak jenis **SkillCard** yang bisa dipegang banyak pemain dan juga di bagian tengah board untuk ketika user menempati **TileCard**. Kelas ini akan memiliki kontrak untuk melakukan pengocokan kartu, melakukan print kartu, mengambil kartu paling atas dan menambahkan kartu serta mengocok kartu dalam deck.

2.8. Kelas Card

Kelas ini merupakan kelas abstrak yang menjadi representasi kartu pada permainan. Kelas ini akan memiliki atribut **cardName** dan **cardDesc** yang bertipe `string`. Dalam kelas ini juga memiliki fungsi yang harus diimplementasikan pada kelas anaknya yaitu **useCard** tujuannya untuk menggunakan polimorfisme dari jenis-jenis kartu dalam kelas ini. Kelas Card ini akan dibagi kegunaanya menjadi dua yaitu satu yang akan digunakan dalam bagian board jika pemain mendarat pada **tileCard** atau pada kelas **skillCard** yang akan digunakan pada **cardDeck** masing-masing pemain.

2.8.1. Kelas AutoUseCard

Kelas ini adalah turunan dari kelas **card** yang termasuk dalam permainan pada bagian board untuk jika pemain mendarat pada **TileCard**.

2.8.1.1. BirthDayCard

Kelas ini adalah turunan dari **AutoUseCard** kelas ini memiliki fungsi spesifik yaitu menjalankan `useCard()` untuk hadiah ulang tahun dan semua player harus membayar M100 kepada pemain yang menggunakan kartu.

2.8.1.2. DoctorCard

Kelas ini adalah turunan dari **AutoUseCard** kelas ini memiliki fungsi spesifik yaitu menjalankan `useCard()` untuk user jatuh sakit dan harus membayar M700.

2.8.1.3. CampaignCard

Kelas ini adalah turunan dari **AutoUseCard** kelas ini memiliki fungsi spesifik yaitu menjalankan `useCard()` untuk user ingin mencalonkan menjadi walikota dan harus membayar M200 ke seluruh Pemain.

2.8.1.4. **NearestStationCard**

Kelas ini adalah turunan dari **AutoUseCard** kelas ini memiliki fungsi spesifik yaitu menjalankan `useCard()` untuk memberikan kesempatan pemain untuk pergi ke stasiun terdekat.

2.8.1.5. **MoveBackCard**

Kelas ini adalah turunan dari **AutoUseCard** kelas ini memiliki fungsi spesifik yaitu menjalankan `useCard()` untuk memaksa pemain untuk mundur 3 petak.

2.8.1.6. **ToJailCard**

Kelas ini adalah turunan dari **AutoUseCard** kelas ini memiliki fungsi spesifik yaitu menjalankan `useCard()` untuk memaksa pemain untuk masuk penjara.

2.8.2. **SkillCard**

Kelas ini adalah turunan dari **card** yang memiliki fungsi utama untuk menjalankan kartu strategis yang hanya dapat dimiliki 3 untuk masing-masing pemain. Kelas digunakan untuk menjalankan kartu skill masing-masing pemain di awal round.

2.8.2.1. **MoveCard**

Kelas ini adalah turunan dari **skillCard** memiliki atribut `moveTileMax` bertipe `int` dan juga fungsi `useCard(Player)` untuk memungkinkan pemain untuk bergerak maju sekian petak serta fungsi `setMoveTileMax()` untuk menentukan nominal langkah pergerakan ditentukan secara acak pada saat pemain pertama kali mendapatkan kartu ini.

2.8.2.2. DiscountCard

Kelas ini adalah turunan dari **skillCard** memiliki atribut `discount` bertipe `int` untuk menyimpan data `discount`. Fungsi yang dimiliki fungsi ini adalah `useCard(Player)` untuk menggunakan kartu ini di awal permainan serta fungsi bersifat `private` `setDiscount()` untuk mengatur `discount` secara acak.

2.8.2.3. ShieldCard

Kelas ini adalah turunan dari **skillCard** kelas ini memiliki fungsi untuk menjalankan `useCard(Player)` untuk menyalakan card effect `SHIELD`.

2.8.2.4. TeleportCard

Kelas ini adalah turunan dari **skillCard**, kelas ini dapat memindahkan player ke seluruh tempat di board dengan fungsi `useCard(player, string Tilecode)`.

2.8.2.5. LassoCard

Kelas ini adalah turunan dari **skillCard**, kelas ini dapat memindahkan player musuh didepan player pemain ke posisi player pemain sekarang dengan `useCard(player)`.

2.8.2.6. DemolitionCard

Kelas ini adalah turunan dari **skillCard**, kelas ini dapat merusak properti lawan dengan fungsi `useCard(player, string Tilecode)`.

2.8.2.7. FreeJailCard

Kelas ini adalah turunan dari **skillCard**, kelas ini memberi kesempatan player untuk langsung keluar dari penjara.

2.9. Kelas DataManager

Kelas ini merupakan kelas yang merepresentasikan Data Access layer yang menangani pembacaan dan penulisan file dari sebuah object DataManager yang dikerjakan. Kelas DataManager memiliki atribut utama:

- a. **configMisc** dengan tipe `string` sebagai representasi nama dari file konfigurasi saldo awal dan max turn yang diberikan pada permainan.
- b. **configProperty** dengan tipe `string` sebagai representasi nama dari file konfigurasi petak properti yang akan dikerjakan dalam konteks memuat pengaturan dari id, kode, nama, jenis warna, harga lahan, nilai gadai, dan harga sewa dari petak kosong hingga hotel.
- c. **configTax** dengan tipe `string` sebagai representasi nama dari file konfigurasi petak pajak yang memuat informasi nilai dari pph flat & presentase, serta nilai dari pbm flat.
- d. **configUtility** dengan tipe `string` sebagai representasi nama dari file konfigurasi fasilitas umum (*utility*) yang mengatur faktor pengali yang diberikan berdasarkan jumlah utility yang dipunyai.
- e. **configRailroad** dengan tipe `string` sebagai representasi nama dari file konfigurasi stasiun/kereta api (*railroad*) yang mengatur biaya sewa aditif yang diberikan berdasarkan jumlah utility yang dipunyai.
- f. **configSpecial** dengan tipe `string` sebagai representasi nama dari file konfigurasi petak spesial yang mendefinisikan uang yang didapat ketika melewati go serta biaya dari keluar penjara.
- g. **configAction** dengan tipe `string` sebagai representasi nama dari file konfigurasi petak aksi (*action*) seperti petak go, dnu, dsb.

Selain atribut, kelas ini juga memiliki metode spesifik untuk mengkonfigurasi load/savenya, yaitu `loadMisc()`, `loadProperty()`, `loadTax()`, `loadUtility()`, `loadRailroad()`, `loadSpecial()`, `loadAction()`, `void loadConfig()` untuk mendefinisikan dan membantu load sebuah file config dan `void load(string)` dan `void save(string, bool = false)` sebagai penanganan untuk membantu save/muat sebuah file. Terakhir, `isFileExist(const string&)` berperan sebagai method helper untuk mengecek apakah ada file dalam direktori yang memiliki nama string tersebut ada.

2.10. Kelas NimonspoliException

NimonspoliException adalah kelas turunan `std::exception` yang bertindak sebagai akar dari seluruh *exception* yang dapat terjadi selama gim Nimonspoli berlangsung. Pendefinisian kelas ini dibutuhkan agar program bisa melakukan *error handling* dalam berbagai aksi yang dilakukan di dalam gim. Atribut yang dimiliki kelas ini:

- a. **errorCode** dengan tipe `int` sebagai pengidentifikasi jenis *exception* yang terjadi, dan
- b. **errorMessage** dengan tipe `string` untuk menyimpan teks yang bisa di-*output* ketika *error* terjadi.

Kelas ini diturunkan ke berbagai kelas *exception* lain yang masing-masing akan dilempar (*throw*) pada kondisi tertentu. Beberapa kelas yang diturunkan memiliki atribut tambahan yang memiliki fungsi *getter*-nya masing-masing.

2.10.1. InvalidDiceException

Kondisi pelemparan: nilai dadu yang invalid saat melakukan perintah ATUR_DADU X Y

Atribut tambahan: **dice1Value** dan **dice2Value** bertipe `int`.

2.10.2. InvalidTileCodeException

Kondisi pelemparan: kode petak yang diberikan ketika menjalankan perintah CETAK_AKTA tidak valid.

Atribut tambahan: **tileCode** bertipe `string`.

2.10.3. NoPropertyException

Kondisi pelemparan: ketika menjalankan perintah CETAK_PROPERTI, namun *player* yang menjalankan tidak memiliki properti apapun.

2.10.4. NotEnoughMoneyException

Kondisi pelemparan: *player* gagal untuk melakukan kewajiban membayarnya yang dapat terjadi pada operasi BAYAR_SEWA, PAJAK, serta KESEMPATAN DAN DANA UMUM sehingga dilanjutkan ke operasi BANGKRUT DAN

AMBIL ALIH ASET. Pembayaran yang bukan kewajiban *player*, seperti pada operasi BELI, TEBUS, BANGUN, dan LELANG, juga dapat menghasilkan *exception* ini, namun tidak diteruskan ke operasi BANGKRUT DAN AMBIL ALIH ASET, melainkan sesuai dengan operasi masing-masing.

Atribut tambahan: **failedToPurchase** bertipe *string*, serta **moneyNeeded** dan **moneyAvailable** bertipe *int*.

2.10.5. **AbilityUsedException**

Kondisi pelemparan: *player* mencoba mengaktifkan kartu kemampuannya (dengan perintah GUNAKAN_KEMAMPUAN), padahal sebelumnya *player* sudah mengaktifkan kartu kemampuan sebelumnya di giliran yang sama.

2.10.6. **AbilityExceededException**

Kondisi pelemparan: gim memberikan kepada *player* kartu kemampuan, namun kartu kemampuan *player* sudah penuh (maksimal 3 kartu di tangan *player*) sehingga memicu operasi DROP KARTU KEMAMPUAN.

2.10.7. **NoPropertyToMortgageException**

Kondisi pelemparan: ketika *player* melakukan perintah GADAI, namun tidak ada properti milik *player* yang bisa digadaikan.

2.10.8. **FailedMortgageException**

Kondisi pelemparan: ketika *player* gagal melakukan perintah GADAI karena properti dengan grup warna yang sama dengan yang ingin digadaikan masih memiliki bangunan.

Atribut tambahan: **propertyName** dan **colorGroup** bertipe *string*.

2.10.9. **NoMortgageException**

Kondisi pelemparan: *player* melakukan perintah TEBUS, namun *player* tidak memiliki properti apapun yang sedang digadaikan.

2.10.10. InvalidChoiceException

Kondisi pelemparan: *player* memasukkan nilai yang tidak dalam pilihan yang ditampilkan pada saat gim meminta pilihan dari *player* yang bisa terjadi pada perintah/operasi BAYAR_PAJAK, GADAI, TEBUS, BANGUN, BANGKRUT DAN AMBIL ALIH ASET, GUNAKAN_KEMAMPUAN, dan DROP KARTU KEMAMPUAN.

Atribut tambahan: **minExpectedChoice**, **maxExpectedChoice**, dan **actualChoice** bertipe `int`.

2.10.11. SaveFailedException

Kondisi pelemparan: ketika perintah SIMPAN gagal dijalankan sampai akhir.

2.10.12. LoadFailedException

Kondisi pelemparan: ketika perintah MUAT gagal dijalankan sampai akhir.

2.11. Kelas UI

Kelas UI merepresentasikan User Interaction layer pada program. Kelas UI akan handle bagaimana sebuah input dihandle, dan juga bagaimana sebuah hasil akan diprint ke layar. Kelas UI memiliki atribut sebagai berikut:

- a. **InputStream** dengan tipe `istream` untuk mengatur input pemain
- b. **OutputStream** dengan tipe `ostream` sebagai sarana tampilan output di layar

kelas ini memiliki *methode* yaitu:

- a. `input(const string& message,`
- b. `function validator, <T>& returnValue) const,`
- c. `output(const <T>& object) const,`
- d. `void cetakPapan(const Board& board),`
- e. `void cetakAkta(const Property& property),`

- f. `void cetakProperti(const Player& player),`
- g. `void cetakLog(const vector<StateLog>& stateLogs.`

3. Justifikasi Desain

Secara umum, kelas didesain berdasarkan tiga lapisan utama untuk mengimplementasikan *layered architecture*. Lapisan tersebut adalah *UI Layer*, *Game Logic Layer*, dan *Data Access Layer*. Pada desain kelas, *Game Logic Layer* menjadi lapisan yang paling mendominasi karena banyak mekanisme serta peraturan yang perlu diimplementasikan dalam gim. Di sisi lain, *UI Layer* hanya berisi satu kelas (UI) yang memiliki fungsi-fungsi *input/output* dasar serta beberapa fungsi hasil implementasi perintah yang dapat dijalankan di gim (CETAK_PAPAN, CETAK_AKTA, CETAK_PROPRTI, CETAK_LOG), sedangkan *Data Access Layer* hanya memiliki dua fungsi yang masing-masing bertugas dalam penyimpanan *state* gim ke dalam suatu *file* dan pemuatan gim ke dalam program dari suatu *file*.

Terdapat perubahan-perubahan kelas diagram yang dilakukan saat implementasi. Perubahan besar antara lain adalah pengimplementasian kelas AuctionManager dan kelas CommandHandler. Hal ini diperlukan karena GameManager sudah memiliki banyak tanggung jawab, sehingga lebih baik beberapa tanggung jawab dari GameManager dipindahkan ke kelas baru. Kelas AuctionManager mengurus proses lelang yang terjadi di gim. Kelas ini perlu diimplementasikan sendiri karena perlu mengatur urutan *bidding* pemain. Kelas CommandHandler mengurus permintaan perintah-perintah yang diberikan pemain serta masukan pemain untuk memenuhi permintaan gim. Ini juga perlu diimplementasikan ke kelas sendiri karena banyak sekali perintah yang dapat dilakukan oleh pemain dan akan lebih terbaca kodenya bila diimplementasikan di kelas terpisah dari GameManager.

Desain kelas berbasis *layered architecture* ini memberikan kejelasan terhadap peran setiap kelas dalam merealisasikan program gim. Dengan pembagian lapisan menjadi *UI Layer*, *Game Logic Layer*, dan *Data Access Layer*, kelas yang menjadi bagian di dalam masing-masing lapisan tersebut menjadi memiliki tanggung jawab yang terbatas sehingga perencanaan implementasi setiap kelas tidak melantur ke mana-mana (contoh, kelas GameManager berada pada *Game Logic Layer*, sehingga tidak perlu mengurus penyimpanan/pemuatan data serta permintaan *input* dan pengeluaran *output* gim). Namun, dengan pemilihan tiga *layer* tersebut, pembagian kelas-kelas yang masuk ke dalam tiap lapisan menjadi tidak merata. Mengingat bahwa gim memiliki peraturan dan mekanisme yang banyak di dalamnya, *Game Logic Layer* menjadi lapisan yang memiliki sangat banyak kelas di dalamnya. Hal ini dapat membuat pendefinisian kelas di dalam *Game Logic Layer* cukup sulit karena harus menghiraukan berbagai kelas di dalam lapisan tersebut yang saling berkaitan.

Beberapa kendala dihadapi ketika mencoba mendesain kelas yang berlandaskan *Object Oriented Programming (OOP)*. Masalah yang paling sering dihadapi adalah dalam penentuan kelas yang sesuai untuk dimasukkan *method* tertentu. Terkadang terdapat *method* yang mengubah *state* beberapa objek yang masing-masing berasal dari kelas yang berbeda sehingga *method* tersebut dinilai valid jika dimasukkan ke dalam salah satu dari beberapa objek tersebut. Kendala lain yang dihadapi adalah ketika mencari asal-usul suatu hal terjadi pada jalannya program. Karena banyak sekali kelas yang diimplementasikan, sehingga banyak sekali kode yang ditulis serta alur logika yang lompat-lompat antarkelas. Hal ini memberatkan karena membuat proses *debugging* sulit untuk mencari mana kesalahan yang terjadi di program.

4. Penerapan Konsep OOP

4.1. Inheritance & Polymorphism

Konsep *Inheritance* dalam pemrograman berbasis objek adalah konsep yang menghubungkan antara dua kelas atau lebih dalam relasi *is-a*. Konsep ini memungkinkan suatu kelas untuk melakukan penurunan/pewarisan atribut serta kontrak kepada kelas lain. Kelas yang menerima pewarisan dalam *inheritance*, disebut sebagai subkelas, akan memiliki atribut dan kontrak yang sama dengan kelas leluhurnya, yang disebut sebagai superkelas.

4.1.1. Kelas Tile

Kelas **Tile** memiliki *inheritance* kepada kelas **Property** dan sejumlah kelas yang menggambarkan petak aksi seperti **CardTile**, **Festival**, **Tax**, **Go**, **Prison**, **Trap**, dan **FreeParking**. Konsep *inheritance* pada kelas-kelas tersebut digunakan untuk menyatukan seluruh jenis petak berbeda sebagai satu petak, hal ini berfungsi juga dalam membentuk kelas **Board** yang merupakan **std::vector** dari kelas **Tile** untuk menggambarkan papan permainan. Berikut adalah potongan kode deklarasi dari kelas **Tile**.

```
class Tile {  
protected:  
    int index;  
    std::string code;  
    std::string name;
```

```

    std::string color;

public:
    Tile(int index, const std::string& code, const std::string& name, const
std::string& color);
    virtual ~Tile() = default;

    int getIndex() const { return index; }
    std::string getCode() const { return code; }
    std::string getName() const { return name; }
    std::string getColor() const { return color; }
    virtual void runTile(Player*) = 0;
    virtual int getAssetValue() const { return 0; }
};

```

Tile.hpp

Setiap atribut yang terdapat pada kelas **Tile** juga dimiliki oleh kelas-kelas turunannya. Selain itu, terdapat juga metode `void runTile(Player*)` serta `int getAssetValue()` yang merupakan metode *pure virtual*, ditandai oleh `= 0` yang terdapat pada deklarasi metode. Selayaknya *pure virtual*, metode tersebut tidak memiliki definisi langsung pada kelas **Tile** namun terdapat pada kelas-kelas turunannya sehingga kedua metode tersebut menjadikan kelas **Tile** kelas abstrak. Berikut adalah potongan kode salah satu kelas yang menjadi *inheritance* dari kelas **Tile**, yaitu kelas **Go** yang menjadi representasi untuk petak mulai

```

class Go : public Tile {
private:

```

```

    int payment;
public:
    Go(int, const std::string&, const std::string& name, const std::string&, int payment);
    void givePayments(Player*);
    void runTile(Player*) override;
};

```

Tile.hpp

4.1.2. Kelas Property

Kelas Property memiliki *inheritance* kepada kelas **Street**, **Utility**, dan **Railroad**. Konsep *inheritance* digunakan pada kelas-kelas tersebut karena memiliki karakteristik yang serupa, yaitu memiliki atribut pemilik serta berkaitan dengan sistem bayar dan sewa lahan namun ketiga kelas tersebut tentunya memiliki *behavior* yang unik. Berikut adalah potongan kode dari deklarasi kelas **Property**

```

class Property : public Tile {
protected:
    int landCost;
    int mortgageValue;
    int festivalMultiplier;
    int festivalDuration;
    Player* owner;
    PROPERTY_STATUS propertyStatus;

public:
    Property(int index, const std::string& code, const std::string& name, const

```

```

std::string& color, int landCost, int mortgageValue, int festivalMultiplier, int
festivalDuration, Player* owner, PROPERTY_STATUS propertyStatus);

    int getLandCost() const;
    int getMortgageValue() const;
    int getFestivalMultiplier() const;
    int getFestivalDuration() const;
    Player* getOwner() const;
    PROPERTY_STATUS getPropertyStatus() const;

    void setFestivalMultiplier(int);
    void setFestivalDuration(int);
    void setOwner(Player*);
    void setPropertyStatus(PROPERTY_STATUS);

    virtual int getRentCost() const = 0;
    virtual int getRentCostLevel(int level) const;
    int getAssetValue() const override;

    // Untuk Memanggil GUI
    virtual void callViewer(PropertyRenderCall& tile) = 0;

};

```

Tile.hpp

Terlihat pada cuplikan kode di atas, kelas **Property** memiliki metode *pure virtual* yaitu `int getRentCost() = 0` yang memberikan jumlah sewa setiap properti yang dimiliki pemain. Metode tersebut juga membuat kelas **Property** sebuah kelas abstrak sehingga tidak dapat diinstansiasi langsung (harus melalui kelas-kelas turunannya). Berikut adalah cuplikan kode deklarasi kelas **Street**, kelas yang merepresentasikan petak-petak bertipe *Street* dalam permainan.

```
class Street : public Property {
private:
    int houseCost;
    int hotelCost;
    std::vector<int> rentCost;
    int currentLevel;

public:
    Street(int index, const std::string& code, const std::string& name, const std::string&
color, int landCost, int mortgageValue, int festivalMultiplier, int festivalDuration,
Player* owner, PROPERTY_STATUS propertyStatus, int houseCost, int hotelCost,
std::vector<int> rentCost, int currentLevel);

    int getHouseCost() const;
    int getHotelCost() const;
    int getCurrentLevel() const;
    void setCurrentLevel(int currentLevel);
    int getBuildCost() const;
    bool canBuild(Player& player) const;
    bool build(Player& player);
    void runTile(Player*) override;
    int getRentCost() const override;
```

```

    int getRentCostLevel(int level) const override;
    int getAssetValue() const override;

    // Untuk interface GUI
    void callViewer(PropertyRenderCall& other) override;
    int getBuildingValue() const;
};

```

Tile.hpp

4.1.3. Kelas Card

Kelas **Card** adalah kelas yang mengatur seluruh mekanisme kartu di dalam permainan, dikarenakan terdapat beberapa tipe kartu dengan karakteristik yang berbeda maka kelas ini memiliki *inheritance* kepada kelas-kelas lain. Kelas yang menjadi turunan dari kelas **Card** adalah kelas **AutoUseCard** dan **SkillCard**. Kedua kelas tersebut memiliki *inheritance*-nya masing-masing yang akan dijelaskan pada bagian selanjutnya. Berikut adalah cuplikan kode deklarasi kelas **Card**.

```

class Card {
protected:
    std::string cardName;
    std::string cardDescription;

public:
    Card(const std::string& cardName, const std::string& cardDescription);
    virtual ~Card() = default;

```

```

std::string getCardName() const;
std::string getCardDescription() const;
virtual void useCard(Player*, std::vector<Player*>) = 0;
};

```

Card.hpp

Kelas **Card** memiliki atribut `cardName` dan `cardDescription` yang dimiliki oleh setiap kelas-kelas turunannya juga. Selain itu, terdapat metode *pure virtual* yaitu `void useCard(Player*, std::vector<Player*>) = 0` yang memiliki algoritma berbeda-beda untuk setiap kelas turunan **Card**. Metode *pure virtual* tersebut juga membuat kelas **Card** menjadi kelas abstrak sehingga tidak dapat diinstansiasi secara langsung tanpa melalui kelas turunannya.

4.1.4. Kelas AutoUseCard

Kelas **AutoUseCard** adalah kelas turunan dari kelas **Card** dan juga memiliki *inheritance* kepada kelas-kelas lain seperti **BirthDayCard**, **DoctorCard**, **CampaignCard**, **NearestStationCard**, **MoveBackCard**, dan **ToJailCard**. Kelas ini dijadikan memiliki *inheritance* karena setiap kartu yang merupakan tipe *auto-use* memiliki karakteristik yang unik. Berikut adalah cuplikan kode deklarasi dari kelas **AutoUseCard** serta kelas-kelas turunannya.

```

class AutoUseCard : public Card {
private:
    CARD_TYPE cardType;
public:
    AutoUseCard(const std::string& cardName, const std::string& cardDescription);
    virtual ~AutoUseCard() = default;

    CARD_TYPE getCardType() const;

```

```
        void useCard(Player*, std::vector<Player*>) override = 0;
    };

    class BirthDayCard : public AutoUseCard {
    public:
        BirthDayCard();
        void useCard(Player*, std::vector<Player*>) override;
    };

    class DoctorCard : public AutoUseCard {
    public:
        DoctorCard();
        void useCard(Player*, std::vector<Player*>) override;
    };

    class CampaignCard : public AutoUseCard {
    public:
        CampaignCard();
        void useCard(Player*, std::vector<Player*>) override;
    };

    class NearestStationCard : public AutoUseCard {
    public:
        NearestStationCard();
        void useCard(Player*, std::vector<Player*>) override;
    };
};
```

```

class MoveBackCard : public AutoUseCard {
public:
    MoveBackCard();
    void useCard(Player*, std::vector<Player*>) override;
};

class ToJailCard : public AutoUseCard {
public:
    ToJailCard();
    void useCard(Player*, std::vector<Player*>) override;
};

```

Card.hpp

4.1.5. Kelas SkillCard

Kelas **AutoUseCard** adalah kelas turunan dari kelas **Card** dan juga memiliki *inheritance* kepada kelas-kelas lain seperti **MoveCard**, **DiscountCard**, **ShieldCard**, **TeleportCard**, **LassoCard**, dan **DemolitionCard**. Kelas ini dijadikan memiliki *inheritance* karena setiap kartu yang merupakan tipe *skill* memiliki karakteristik yang unik. Berikut adalah cuplikan kode deklarasi dari kelas **SkillCard** serta kelas-kelas turunannya.

```

class SkillCard : public Card {
protected:
    int cardValue;
    int cardDuration;
public:
    SkillCard(const std::string& cardName, const std::string& cardDescription, const int

```

```

cardValue, const int cardDuration);
    virtual ~SkillCard() = default;

    CARD_TYPE getCardType() const;
    int getCardValue() const;
    int getCardDuration() const;
    void setCardValue(int value);
    void setCardDuration(int duration);
    void useCard(Player*, std::vector<Player*>) override = 0;
};

class MoveCard : public SkillCard {
public:
    MoveCard();
    MoveCard(int);
    int getTileCount() const;
    void setTileCount(int);
    void useCard(Player*, std::vector<Player*>) override;
};

class DiscountCard : public SkillCard {
public:
    DiscountCard();
    DiscountCard(int);
    float getDiscount() const;
    void setDiscount(int);
    void useCard(Player*, std::vector<Player*>) override;
};

```

```
};

class ShieldCard : public SkillCard {
public:
    ShieldCard();
    void useCard(Player*, std::vector<Player*>) override;
};

class TeleportCard : public SkillCard {
public:
    TeleportCard();
    void useCard(Player*, std::vector<Player*>) override;
};

class LassoCard : public SkillCard {
public:
    LassoCard();
    void useCard(Player*, std::vector<Player*>) override;
};

class DemolitionCard : public SkillCard {
public:
    DemolitionCard();
    void useCard(Player*, std::vector<Player*>) override;
};

class FreeJailCard : public SkillCard {
```

```

public:
    FreeJailCard();
    void useCard(Player*, std::vector<Player*>) override;
};

```

4.2. Method/Operator Overloading

Operator overloading merupakan konsep dalam bahasa pemrograman, terutama C++, yang mengizinkan pendefinisian ulang operator (seperti + - * = << >>) agar dapat digunakan sesuai dengan karakteristik kelas tertentu. Pendefinisian ulang ini berfungsi untuk memudahkan penulisan dan penambahan kode dengan *syntax* yang lebih intuitif. *Operator overloading* ditulis dengan nama metode `operatorX` dengan X yaitu jenis operator yang didefinisikan ulang, metode ini memiliki dua jenis, yaitu uner dan biner yang menggambarkan jumlah argumen yang dibutuhkan.

Di dalam algoritma yang diimplementasikan, konsep *operator overloading* digunakan pada kelas **Player** dan berfungsi untuk melakukan perubahan *currency* milik *player* secara alami. Secara spesifik, kelas **Player** memiliki metode `operator+=(int money)` dan `operator-=(int money)` dengan fungsi untuk menambahkan dan mengurangi uang milik **Player** sebanyak parameter *money*. Berikut adalah cuplikan kode yang menunjukkan konsep *operator overloading* pada kelas **Player**.

```

Player* Player::operator+=(int money) {
    this->currency += money;
    return this;
}

Player* Player::operator-=(int money) {
    this->currency -= money;
    return this;
}

```


}

Player.cpp

4.3. Template & Generic Classes

Di dalam algoritma yang diimplementasikan, konsep *template* & generic classes digunakan pada kelas **CardDeck** dan berfungsi untuk menyimpan card-card yang saat ini sudah dipakai dan card yang saat ini sudah ada. Fungsi template pada class **CardDeck** adalah untuk fleksibilitas dari sifat-sifat card tertentu yang memiliki tipe berbeda-beda. Sehingga dari card yang memiliki sifat-sifat yang berbeda tersebut dapat digeneralisasikan / dikelompokkan menjadi satu kesatuan deck card yang mudah dikelompokkan (karena memakai template class).

```
template <typename T>
class CardDeck {
private:
    std::vector<T*> availableCards;
    std::vector<T*> usedCard;

public:
    CardDeck() = default;
    CardDeck(CardDeck&& noexcept) = default;
    CardDeck& operator=(CardDeck&& noexcept) = default;
    virtual ~CardDeck() = default;

    void addCard(T* card);
    void shuffleDeck();
    void printDeck() const;
```

```

T* topDeck();
T* getRandomCard();
T* removeAt(size_t index);
size_t size() const;
bool empty() const;
const std::vector<T*>& getCards() const;
};

```

Deck.hpp

4.4. Exception

Exception yang diimplementasikan merupakan sebuah mekanisme untuk menangani masalah yang muncul saat program dijalankan (runtime), seperti input yang invalid, unexpected behavior, atau file yang tidak ditemukan. Tujuannya adalah agar program tidak langsung "crash" atau berhenti tiba-tiba, melainkan memberikan kesempatan bagi program untuk menangani kesalahan tersebut secara elegan. Implementasi exception yang digunakan mengikuti format seperti berikut.

```

#pragma once

#include <exception>
#include <string>

using namespace std;

class NimonspoliException : public exception {
private:
    int errorCode;

```

```

        string errorMessage;
    public:
        NimonspoliException(int code, string message) : errorCode(code), errorMessage(message) {}
        int getErrorCode() const noexcept { return errorCode; }
        string getErrorMessage() const noexcept { return errorMessage; }
        virtual const char* what() const noexcept override { return errorMessage.c_str(); }
};

class InvalidDiceException : public NimonspoliException {
    private:
        int dice1Value;
        int dice2Value;
    public:
        InvalidDiceException(int dice1, int dice2) : NimonspoliException(1, "DICE_VALUE_INVALID"),
dice1Value(dice1), dice2Value(dice2) {}
        int getDice1() const noexcept { return dice1Value; }
        int getDice2() const noexcept { return dice2Value; }
};

class InvalidTileCodeException : public NimonspoliException {
    private:
        string tileCode;
    public:
        InvalidTileCodeException(string tileCode) : NimonspoliException(2, "TILE_CODE_INVALID"),
tileCode(tileCode) {}
        string getTileCode() const noexcept { return tileCode; }
};

```

```

class NoPropertyException : public NimonspoliException {
    public:
        NoPropertyException() : NimonspoliException(3, "NO_PROPERTY_ERROR") {}
};

class NotEnoughMoneyException : public NimonspoliException {
    private:
        string failedToPurchase;
        int moneyNeeded;
        int moneyAvailable;
    public:
        NotEnoughMoneyException(string purchaseWhat, int need, int avail) : NimonspoliException(4,
"NOT_ENOUGH_MONEY"), failedToPurchase(purchaseWhat), moneyNeeded(need), moneyAvailable(avail) {}
        string purchaseWhat() const noexcept { return failedToPurchase; }
        int getMoneyNeeded() const noexcept { return moneyNeeded; }
        int getMoneyAvailable() const noexcept { return moneyAvailable; }
};

class AbilityUsedException : public NimonspoliException {
    public:
        AbilityUsedException() : NimonspoliException(5, "SKILLCARD_ALREADY_USED") {}
};

class AbilityExceededException : public NimonspoliException {
    public:
        AbilityExceededException() : NimonspoliException(6, "SKILLCARD_EXCEEDED_CAP") {}
};

```

```

};

class NoPropertyToMortgageException : public NimonspoliException {
    public:
        NoPropertyToMortgageException() : NimonspoliException(7, "NO_PROPERTY_TO_MORTGAGE") {}
};

class FailedMortgageException : public NimonspoliException {
    private:
        string propertyName;
        string colorGroup;
    public:
        FailedMortgageException(string prop, string color) : NimonspoliException(8, "MORTGAGE_FAILED"),
propertyName(prop), colorGroup(color) {}
        string getPropertyName() const noexcept { return propertyName; }
        string getColorGroup() const noexcept { return colorGroup; }
};

class NoMortgageException : public NimonspoliException {
    public:
        NoMortgageException() : NimonspoliException(9, "NO_MORTGAGED_PROPERTY") {}
};

class InvalidChoiceException : public NimonspoliException {
    private:
        int minExpectedChoice;
        int maxExpectedChoice;
};

```

```

        int actualChoice;
    public:
        InvalidChoiceException(int selected, int min, int max) :
            NimonspoliException(10, "INVALID_CHOICE"), minExpectedChoice(min), maxExpectedChoice(max),
actualChoice(selected) {}
        int getMinChoice() const noexcept { return minExpectedChoice; }
        int getMaxChoice() const noexcept { return maxExpectedChoice; }
        int getActualChoice() const noexcept { return actualChoice; }
};

class SaveFailedException : public NimonspoliException {
    public:
        SaveFailedException() : NimonspoliException(11, "SAVE_FAILED") {}
};

class LoadFailedException : public NimonspoliException {
    public:
        LoadFailedException() : NimonspoliException(12, "LOAD_FAILED") {}
};

class FileExistsException : public NimonspoliException {
    private:
        string fileName;
    public:
        FileExistsException(const string& fileName) : NimonspoliException(13, "FILE_EXISTS"),
fileName(fileName) {}
        const string& getFileName() const noexcept { return fileName; }
};

```

```
};

class FileNotExistsException : public NimonspoliException {
    private:
        string fileName;
    public:
        FileNotExistsException(const string& fileName) : NimonspoliException(14, "FILE_NOT_EXISTS"),
fileName(fileName) {}
        const string& getFileName() const noexcept { return fileName; }
};

class SaveProhibitedException : public NimonspoliException {
    public:
        SaveProhibitedException() : NimonspoliException(15, "SAVE_PROHIBITED") {}
};

GameException.hpp
```

Cuplikan kode berikut menunjukkan salah satu implementasi *exception* yaitu pada pengolahan perintah SIMPAN. Cuplikan kode tersebut menunjukkan bagaimana metode `DataManager::save()` melakukan `throw` berbagai *exception* untuk setiap kasus yang mungkin terjadi, kemudian bagaimana metode `CommandHandler::execute()` melakukan `try catch()` untuk menangani

```
void DataManager::save(string fileName, bool override) {
    GameManager& game = GameManager::getInstance();
```

```

const std::vector<StateLog>& logs = game.getLogger().getLogs();
if(!logs.empty() && logs.back().getTurn() == game.getTurn()) {
    throw SaveProhibitedException();
}

string path = "data/" + fileName;
if(isFileExists(path) && !override) {
    throw FileExistsException(path);
}
ofstream file("data/" + fileName);
if (!file.is_open()) {
    throw SaveFailedException();
}

```

DataManager.cpp

```

else if (command == "SIMPAN") {
    string first;
    if (iss >> first) {
        DataManager dm;
        bool isOverride = false;
        while(true) {
            try {
                game.writeLine("Menyimpan permainan...");
                dm.save(first, isOverride);
                game.writeLine("Permainan berhasil disimpan ke: " + first);
            }
        }
    }
}

```



```

        } catch (SaveProhibitedException& e) {
            game.WriteLine("Command SIMPAN tidak bisa dilakukan jika telah melakukan suatu
aksi.");
        } catch (FileExistsException& e) {
            game.WriteLine("File \"" + e.GetFileName() + "\" sudah ada.");
            string confirmation = askInput("Timpa file lama? (y/n): ");
            if(confirmation == "y" || confirmation == "Y") {
                isOverride = true;
                continue;
            }
            game.WriteLine("SIMPAN dibatalkan.");
        } catch (SaveFailedException& e) {
            game.WriteLine("Gagal menyimpan file! Pastikan direktori dapat ditulis.");
        }
        break;
    }
}

else {
    game.WriteLine("Format: SIMPAN <nama_file>.txt");
}

```

CommandHandler.cpp

4.5. C++ Standard Template Library

C++ Standard Template Library (STL) adalah pustaka yang menyediakan struktur data, yaitu *container*, serta algoritma generik yang dapat membantu penulisan algoritma secara ringkas. Algoritma yang terdapat pada STL merupakan bawaan dari *compiler* C++ sehingga untuk mengaksesnya memerlukan definisi *include library* bawaan seperti `#include <algorithm>` atau `#include <vector>` serta penggunaannya membutuhkan pemanggilan melalui `std::`.

Dalam algoritma *nimonspoli* yang sudah diimplementasikan, penggunaan STL sebagai struktur data dilakukan pada kelas **CardDeck**, **Board**, dan **GameManager**. Serupa dengan penggunaan sebagai struktur data, penggunaan STL sebagai fungsi algoritma generik juga terdapat pada kelas-kelas tersebut. Konsep STL digunakan pada kelas-kelas tersebut untuk mengurangi tanggung jawab setiap kelas sehingga kelas-kelas tersebut hanya menjadi *container*, tidak perlu memikirkan alokasi dan dealokasi memori. Berikut adalah cuplikan kode dari kelas **Board** yang menunjukkan penggunaan STL

```
class Board {
private:
    std::vector<Tile*> tiles;
    int tileCount;
    int goIndex;
    int jailIndex;

public:
    Board(int size);

    Tile* getTile(std::string code) const; // done
    Tile* getTile(int index) const; // done
    Tile* getJailTile() const; // done
    std::vector<Tile*> getTiles() const{return this->tiles;}; // done
    Tile* goToTile(Tile& current, int moveAmount) const; // done
    Tile* goToTile(const std::string&, int);
    void setGoIndex(int index){ this->goIndex = index; } // done
```

```

void setJailIndex(int index){ this->jailIndex = index; } // done
std::vector<Tile*> getColorGroup(const std::string& color) const ; // done
std::vector<Street*> getAllStreet() const;
std::vector<Railroad*> getAllRailroad() const ; // done
std::vector<Utility*> getAllUtility() const ; // done
bool canUpgradeProperty(Player &player, Tile* tile);
int getRailroadLevel(Tile* tile);
int getUtilityLevel(Tile* tile);
Player* getNextPlayer(Player* player);
int stringToIndex(const std::string&);
void addTile(Tile* newTile); // done
void advanceFestivalEffects(Player* owner);
void printBoard() const;
void cetakAkta(std::string code);
};

```

Board.hpp

```

Tile* Board::getTile(string code) const {
    auto it = std::find_if(tiles.begin(), tiles.end(), [&code](const Tile* tile){
        return tile != nullptr && tile->getCode().compare(code) == 0;
    });

    if (it != tiles.end()) {
        return *it;
    } else {

```

```

        return nullptr;
    }
}

std::vector<Tile*> Board::getColorGroup(const std::string& color) const {
    std::vector<Tile*> result;
    std::copy_if(tiles.begin(), tiles.end(), std::back_inserter(result), [&color](const Tile* tile){
        return tile != nullptr && tile->getColor().compare(color) == 0;
    });

    return result;
}

bool Board::canUpgradeProperty(Player &player, Tile* tile) {
    if(tile == nullptr) return false;
    if(tile->getColor() == "Default") return false; // asumsi namanya "Default"
    std::vector<Tile*> colorGroup = getColorGroup(tile->getColor());
    Street* targetProp = dynamic_cast<Street*>(tile);
    if(targetProp->getCurrentLevel() == 5) return false;
    bool imbalanceTileExists = std::any_of(colorGroup.begin(), colorGroup.end(), [&player,
targetProp](Tile* tile){
        Street* prop = dynamic_cast<Street*>(tile);
        if(prop->getOwner() != &player) return true;
        return targetProp->getCurrentLevel() - prop->getCurrentLevel() > 0;
    });
    return !imbalanceTileExists;
}

```

Board.cpp

4.6. Konsep OOP lain

4.6.1. Card Abstract Class

Penggunaan Abstract base class pada kelas card dengan adanya Fungsi Pure Virtual useCard(). Hal ini memastikan semua kartu memiliki implementasi dan tidak dapat diinstansiasi dengan tipe yang telah ditentukan

```
class Card {
protected:
    std::string cardName;
    std::string cardDescription;

public:
    Card(const std::string& cardName, const std::string& cardDescription);
    virtual ~Card() = default;

    std::string getCardName() const;
    std::string getCardDescription() const;
    virtual void useCard(Player*, std::vector<Player*>) = 0;
};
```

4.6.2. Singleton

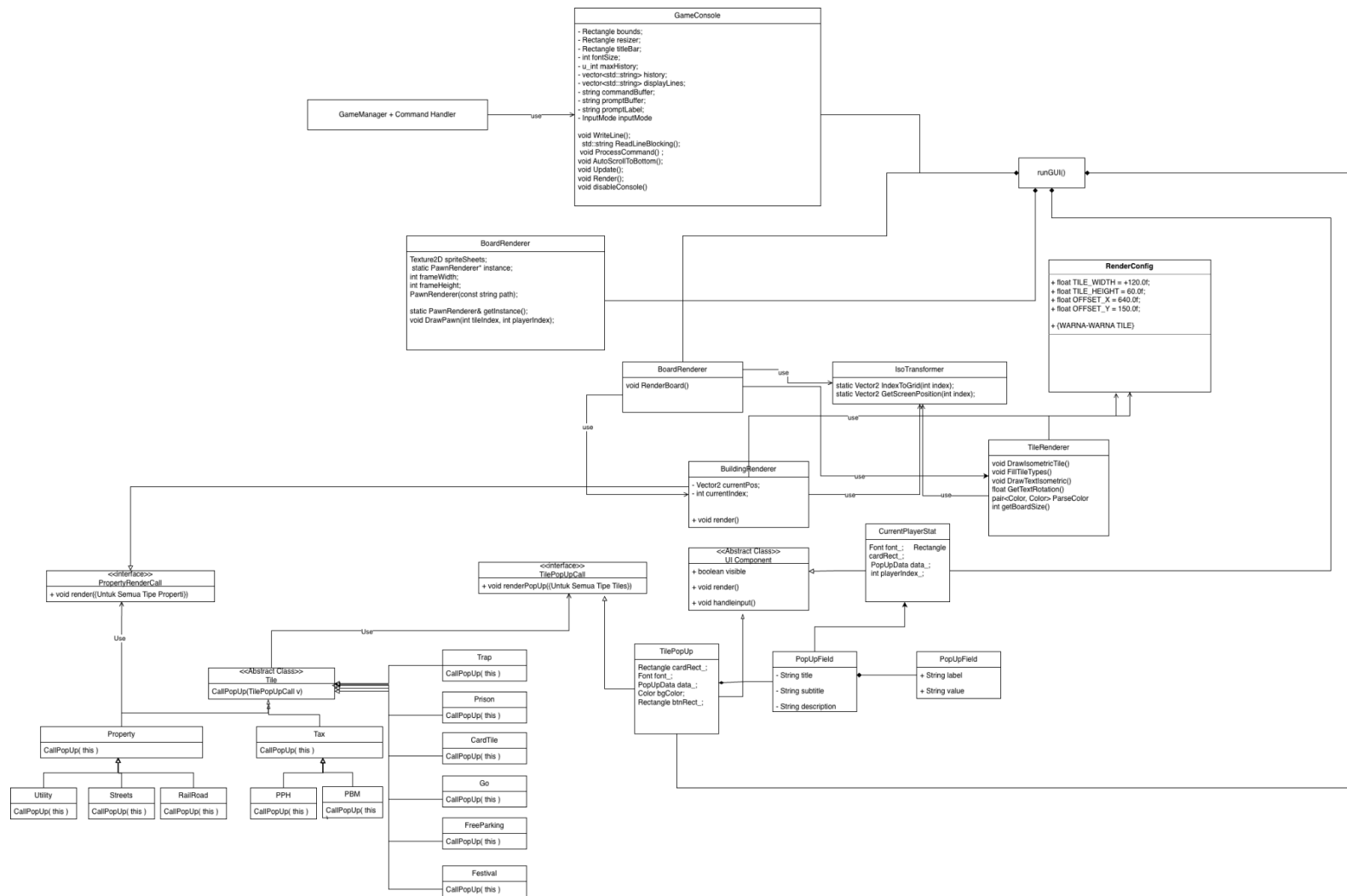
Penggunaan singleton untuk reference ke satu object global dengan getter yang dapat dipanggil secara statik, hal ini diterapkan pada kelas GameManager karena hanya ada objek yang terinstansiasi dalam runtime dan digunakan oleh semua objek lainnya.

```
static GameManager& getInstance() {  
    static GameManager instance;  
    return instance;  
}
```

5. Bonus Yang dikerjakan

5.1. Bonus yang diusulkan oleh spek

5.1.1. GUI



Berikut adalah gambaran implementasi dari GUI pada nimons poli ini, secara umum implementasi GUI ini merupakan pembelajaran inheritance dan polymorfisme yang paling berharga yang dipelajari. Di sini diimplementasikan interface class pada C++ dengan menggunakan abstract class dan penggunaan double dispatch yaitu penggunaan dynamic

binding dan juga overloading method untuk memanfaatkan polymorphisme pada pembuatan rendering building dan juga dalam pemanggilan pop up masing-masing tile. Hasilnya ketika board bisa melakukan rendering property tile berdasarkan tipe turunannya. Pemanggilan pop-up untuk masing-masing tipe tiles yang berbeda. Untuk detail dalam implementasi headernya sebagai berikut

5.1.1.1. RenderConfig

Kelas renderConfig ini berisi konfigurasi dari ukuran masing-masing tile yang akan di render secara isometrik dan juga warna pilihan yang akan digunakan dalam permainan nimonspoli ini, tujuannya agar pemilihan warna menjadi konsisten dan juga ukuran dari tiles tetap konsisten.

```
#pragma once

#include <utility>

#undef PINK
#undef ORANGE

#define COKLAT Color{139, 94, 60, 255}
#define COKLAT_SHADE Color{101, 63, 35, 255}
#define BIRU_MUDA Color{126, 200, 227, 255}
#define BIRU_MUDA_SHADE Color{78, 155, 185, 255}
#define MERAH_MUDA Color{244, 167, 185, 255}
#define MERAH_MUDA_SHADE Color{204, 112, 138, 255}
#define JERUK Color{242, 145, 74, 255}
#define JERUK_SHADE Color{193, 99, 35, 255}
#define MERAH Color{224, 85, 85, 255}
```



```

#define MERAH_SHADE Color{168, 45, 45, 255}
#define KUNING Color{245, 208, 96, 255}
#define KUNING_SHADE Color{195, 157, 38, 255}
#define HIJAU Color{114, 196, 114, 255}
#define HIJAU_SHADE Color{65, 143, 65, 255}
#define BIRU_TUA Color{46, 95, 158, 255}
#define BIRU_TUA_SHADE Color{22, 55, 105, 255}
#define BOARD_BASE Color{235, 225, 210, 255}
#define BOARD_LINE Color{200, 184, 154, 255}
#define BACKGROUND Color{235, 225, 210, 255}
#define ABU Color{ 150, 145, 138, 255 }
#define ABU_SHADE Color{ 90, 86, 80, 255 }
#define BOARD_COKLAT Color{ 193, 154, 107, 255 }

struct RenderConfig {
    // Ukuran satu petak dasar (Rasio Isometrik biasanya 2:1)
    static constexpr float TILE_WIDTH = 120.0f;
    static constexpr float TILE_HEIGHT = 60.0f;

    // Offset agar papan berada di tengah layar (Sesuaikan dengan resolusi window
    // Anda)
    static constexpr float OFFSET_X = 640.0f;
    static constexpr float OFFSET_Y = 150.0f;
};

```

Rende

5.1.1.2. IsoTransformer

IsoTransformer ini adalah implementasi melakukan transformasi pada awalnya tiles yang disimpan dalam bentuk array kontigu menjadi sebuah transformasi menjadi bentuk isometrik dengan transformasi aljabar linier dalam membuat ilusi 3 dimensi. IndexToGrid ini akan mentransformasikan ukuran mana tiles harus dirender dan getScreenPosition() akan mentransformasikan index ke posisi grid yang sudah dibuat isometrik.

```
#pragma once
#include "../RenderConfig.hpp"
#include "raylib.h"

struct RenderConfig;

class IsoTransformer {
public:
    static Vector2 IndexToGrid(int index);
    static Vector2 GetScreenPosition(int index);
};

inline Vector2 operator+(Vector2 v1, Vector2 v2)
{
    return Vector2{v1.x + v2.x, v1.y + v2.y};
}

inline Vector2 operator-(Vector2 v1, Vector2 v2)
{
    return Vector2{v1.x - v2.x, v1.y - v2.y};
}
```

```

}
inline Vector2 operator*(Vector2 v1, float scalar)
{
    return Vector2{v1.x * scalar, v1.y * scalar};
}
inline Vector2 operator/(Vector2 v1, float scalar)
{
    return Vector2 { v1.x / scalar, v1.y / scalar };
}

```

IsoTransformer.hpp

5.1.1.3. TileRenderer

Kelas TileRenderer ini berfungsi dalam melakukan menggambar tile secara isometrik dengan batasan dan urutan tile tertentu dengan tipe tile yang berbeda. Tile Streets disini akan diwarnai sebagian saja dan tile yang lain akan diwarnai dengan penuh. Di sini juga bertugas dalam melakukan transformasi rotasi dari sebuah index ke dalam koordinat isometrik.

```

#pragma once

#include "../RenderConfig.hpp"
#include "../../core/Tile.hpp"
#include "raylib.h"
#include "../include/gui/MathCore/IsoTransformer.hpp"
#include "../include/core/GameManager.hpp"
#include <string>

```

```

class TileRenderer {
public:
    static void DrawIsometricTile(Vector2 centerPos, std::pair<Color, Color>
tileColors);
    static void FillTileTypes(Vector2 tilePos, int index, std::pair<Color, Color>
tileColors);
    static void DrawTextIsometric(const std::string text, Vector2 originOffset, Vector2
tilePos, float rotation);
    static float GetTextRotation(int index);
    // Melakukan parsing tile dan menghasilkan warna utama dan shadenya
    static std::pair<Color, Color> ParseColor(Tile *tile);
    static int getBoardSize() { return
GameManager::getInstance().getBoard().getTiles().size(); }
};

```

TileRenderer.hpp

5.1.1.4. BoardRenderer

Kelas BoardRenderer ini akan memanfaatkan TileRenderer untuk melakukan penggambaran tile dalam screen dengan ukuran tertentu disini juga akan memanggil building renderer untuk menggambarkan masing-masing tipe building untuk property tiles.

```

#pragma once
#include "../NimonspoliGUI.hpp"
#include "../../core/Board.hpp"

```

```
class BoardRenderer {
public:
    void RenderBoard(Board board);
};
```

BoardRenderer.hpp

5.1.1.5. PropertyRenderCall

Kelas PropertyRenderCall ini adalah sebuah interface yang dipakai dalam melakukan double dispatch pemanggilan penggambaran tipe-tipe bangunan pada tiles. Jadi masing-masing turunan dari Property Tiles akan memiliki kontrak untuk memanggil PropertyRenderCall yang akan memanggil render berdasarkan tipenya. Penggunaan ini dipakai agar tidak menggunakan terlalu banyak type casting yang tidak aman.

```
class PropertyRenderCall{
public:
    ~PropertyRenderCall() = default;
    virtual void render(Street* s) = 0;
    virtual void render(Railroad* r) = 0;
    virtual void render(Utility* u) = 0;
};
```

Tile.hpp

5.1.1.6. BuildingRenderer

BuildingRenderer ini adalah turunan dari PropertyRenderCall dan memiliki kontrak untuk mengambarkan masing-masing tipe-tipe bangunan yang akan dipanggil.

```
#include "../include/core/Tile.hpp"

class BuildingRenderer : public PropertyRenderCall {
private:
    Vector2 currentPos;
    int currentIndex;
    void drawHouses(int count);
    void drawStation();
    void drawIsometricHouse(Vector2 IsoPos);
    void drawIsometricStation(Vector2 isoPos);
    void drawIsometricHotel(Vector2 IsoPos);

public:
    BuildingRenderer() : currentPos({0,0}), currentIndex(0) {};
    void setContext(int index);
    void render(Street *s) override;
    void render(Railroad *r) override;
    void render(Utility *u) override;
};
```

BuildingRendererer.hpp

5.1.1.7. TilePopUpCall

TilePopUpCall adalah sebuah virtual abstract class yang digunakan sebagai interface polymorphism dalam pemanggilan PopUp masing masing tipe tiles. Pada tiles sebagai super class akan menurunkan kontrak untuk memanggil masing-masing tile pop up sesuai tipenya. Sehingga masing-masing tile akan memiliki bentuk pop-up yang berbeda.

```
class TilePopUpCall {
public:
    virtual ~TilePopUpCall() = default;
    // Properti
    virtual void renderPopUp(Street *s) = 0;
    virtual void renderPopUp(Railroad *r) = 0;
    virtual void renderPopUp(Utility *u) = 0;

    // Taxes
    virtual void renderPopUp(PBM *pbm) = 0;
    virtual void renderPopUp(PPH *pph) = 0;

    // Special Tiles
    virtual void renderPopUp(CardTile *ct) = 0;
    virtual void renderPopUp(Festival *f) = 0;
    virtual void renderPopUp(Go *g) = 0;
    virtual void renderPopUp(Prison *p) = 0;
    virtual void renderPopUp(Trap *tr) = 0;
    virtual void renderPopUp(FreeParking *fp) = 0;
};
```

UIComponent.hpp

5.1.1.8. UIComponent

UIComponent adalah kelas virtual abstract class yang digunakan sebagai kontrak dasar pembuatan komponen UI yang akan ditampilkan dan berinteraksi dengan user.

```
class UIComponent {
protected:
    bool visible = false;

public:
    virtual ~UIComponent() = default;
    virtual void render() = 0;
    virtual void handleInput() = 0;
    void setVisible(bool status) { visible = status; }
    bool isVisible() const { return visible; }
};
```

UIComponent.hpp

5.1.1.9. TilePopup

TilePopup adalah kelas yang memiliki dua kontrak yang diturunkan oleh UIComponent dan TilePopUpCall dua kontrak ini harus dipenuhi TilePopup karena berfungsi berinteraksi dengan user dan harus menangani pemanggilan tile untuk polymorfisme.

```
class TilePopup : public UIComponent, public TilePopUpCall {
private:
```



```

static constexpr int POPUP_W = 360;
static constexpr int POPUP_H = 480;
static constexpr int PADDING = 20;
static constexpr int HEADER_H = 60;
static constexpr int BTN_W = 110;
static constexpr int BTN_H = 38;

// font size
static constexpr float FS_TITLE = 22.0f;
static constexpr float FS_SUBTITLE = 14.0f;
static constexpr float FS_FIELD = 16.0f;
static constexpr float FS_BODY = 15.0f;
static constexpr float SPACING = 1.0f;

Rectangle cardRect_;
Font font_;
PopUpData data_;
Color bgColor;
Rectangle btnRect_;
void drawPopUp();
void updateButtonRect();
void drawFields(int x, int &cursorY, int maxW);
void drawParagraph(int x, int &cursorY, int maxW, int bottomLimit);

// Text helpers
std::string truncateToFit(const std::string &text,
                          float fontSize, int maxWidth) const;

```

```

// Wrap teks jadi vector baris, tidak overflow maxWidth
std::vector<std::string> wrapText(const std::string &text,
                                float fontSize, int maxWidth) const;

// Ukur teks dengan font_ dan fontSize
Vector2 measure(const std::string &text, float fontSize) const;

public:
    TilePopup();
    void render() override;
    void handleInput() override;
    void renderPopUp(Street *s) override;
    void renderPopUp(Railroad *r) override;
    void renderPopUp(Utility *u) override;

    // Taxes
    void renderPopUp(PBM *pbm) override;
    void renderPopUp(PPH *pph) override;

    // Special Tiles
    void renderPopUp(CardTile *ct) override;
    void renderPopUp(Festival *f) override;
    void renderPopUp(Go *g) override;
    void renderPopUp(Prison *p) override;
    void renderPopUp(Trap *tr) override;
    void renderPopUp(FreeParking *fp) override;

```

```
void setFont(Font font) { font_ = font; }
};
```

UIComponent.hpp

5.1.1.10. PopUpField

PopUpField ini adalah class yang digunakan sebagai field dari pop up sehingga popup dapat menampilkan field untuk element-element yang harus ditampilkan dalam sebuah tabel misalnya seperti biaya dan harganya pada tiles.

```
class PopUpField {
private:
    std::string label_;
    std::string value_;

public:
    PopUpField() = default;
    PopUpField(std::string label, std::string value)
        : label_(std::move(label)), value_(std::move(value)) {}

    // Getters
    const std::string &getLabel() const { return label_; }
    const std::string &getValue() const { return value_; }
```

```
// Setters
void setLabel(std::string label) { label_ = std::move(label); }
void setValue(std::string value) { value_ = std::move(value); }
};
```

UIComponent.hpp

5.1.1.11. PopUpData

PopUpData adalah class Data Umum dari sebuah popUp yang memiliki-atribut-atribut dalam pembuatan pop up untuk masing-masing pop-up tiles tertentu.

```
class PopUpData {
private:
    std::string title_;
    std::string subtitle_;
    std::vector<PopUpField> fields_;
    std::string description_;

public:
    PopUpData() = default;
    PopUpData(std::string title, std::string subtitle,
               std::vector<PopUpField> fields, std::string description)
        : title_(std::move(title)), subtitle_(std::move(subtitle)),
          fields_(std::move(fields)), description_(std::move(description)) {}
};
```

```

// Getters
const std::string &getTitle() const { return title_; }
const std::string &getSubtitle() const { return subtitle_; }
const std::vector<PopUpField> &getFields() const { return fields_; }
const std::string &getDescription() const { return description_; }

// Setters
void setTitle(std::string title) { title_ = std::move(title); }
void setSubtitle(std::string subtitle) { subtitle_ = std::move(subtitle); }
void setDescription(std::string description) { description_ =
std::move(description); }

// Fields – manipulasi koleksi
void setFields(std::vector<PopUpField> fields) { fields_ = std::move(fields); }
void addField(PopUpField field) { fields_.emplace_back(std::move(field)); }
void clearFields() { fields_.clear(); }
bool hasFields() const { return !fields_.empty(); }
bool hasDescription() const { return !description_.empty(); }
};

```

UIComponent.hpp

5.1.1.12. CurrentPlayerStats

Kelas CurrentPlayerStats diimplementasikan untuk menampilkan status dari player yang sedang memiliki giliran.

```

class CurrentPlayerStats : public UIComponent {
private:
    static constexpr int CARD_W = 200;
    static constexpr int CARD_H = 130;
    static constexpr int PADDING = 12;
    static constexpr int MARGIN = 16;

    static constexpr float FS_NAME = 18.0f;
    static constexpr float FS_FIELD = 14.0f;
    static constexpr float SPACING = 1.0f;

    Font font_;
    Rectangle cardRect_;
    PopUpData data_;
    int playerIndex_;

    // Helper - build data_ dari Player
    void syncFromPlayer(const Player &player);
    void drawCard();

    // reuse helper dari TilePopup
    Vector2 measure(const std::string &text, float fontSize) const;
    std::string truncateToFit(const std::string &text,
                             float fontSize, int maxWidth) const;

    static std::string statusToString(PLAYER_STATUS s);
    static std::string effectToString(CARD_EFFECT e, int turns);

```

```

public:
    CurrentPlayerStats(int playerIndex);

    void setFont(Font font) { font_ = font; }

    void update(const Player *player);
    void render() override;
    void handleInput() override {}
};

```

UIComponent.hpp

5.1.1.13. GameConsole

GameConsole adalah kelas yang akan menggunakan implementasi sesuai dari sebuah spek diberikan yaitu input program dalam bentuk CLI sehingga di sini disesuaikan dalam bentuk game console yang dapat menerima input dari user untuk melakukan aksi permainan.

```

#include "raylib.h"
#include <vector>
#include <string>
#include <functional>

class GameConsole
{

```

```

private:
    enum class InputMode {
        Command,
        Prompt
    };

    // Konfigurasi Area
    Rectangle bounds;
    Rectangle resizer;
    Rectangle titleBar;
    int fontSize;
    u_int maxHistory;

    // Data Teks
    std::vector<std::string> history;
    std::vector<std::string> displayLines; // Wrapped lines for rendering

    // std::string currentInput;
    std::string commandBuffer;
    std::string promptBuffer;
    std::string promptLabel;
    InputMode inputMode = InputMode::Command;

    int scrollOffset = 0;
    // bool waitingForBlockingInput = false;
    bool blockingInputReady = false;
    std::string blockingInputResult;

```



```

std::vector<std::string> WrapText(const std::string& text, float maxWidth);

// State Input
bool isActive = true;
int framesCounter = 0;

// Resizing State
bool isResizing = false;
bool isMoving = false;
Vector2 lastMouse = {0.0f, 0.0f};
const float minWidth = 200.0f;
const float minHeight = 150.0f;
const float maxWidth = 1000.0f;
const float maxHeight = 700.0f;
const float titleBarHeight = 25.0f;

std::function<void(std::string)> commandCallback = nullptr;
std::function<void()> blockingRenderCallback = nullptr;

public:
    GameConsole(Rectangle area, int fSize = 12);
    void WriteLine(std::string text);
    std::string ReadLineBlocking(const std::string& prompt);
    void ProcessCommand(std::string cmd) ;
    void AutoScrollToBottom();
    void Update();

```

```

void Render();
void disableConsole() {
    this->isActive = false;
}

void SetCommandCallback(std::function<void(std::string)> callback) {
    commandCallback = callback;
}

void SetBlockingRenderCallback(std::function<void()> callback) {
    blockingRenderCallback = callback;
}

Rectangle getBounds() {return bounds;};
~GameConsole();
};

```

GameConsole.hpp

5.1.2. Papan Dinamis

Pada implementasi, papan direpresentasikan oleh kelas Board yang menggunakan objek `vector` untuk menyimpan daftar petak yang ada pada papan. Daftar petak pada board, karena menggunakan `vector`, bisa di-*resize* sesuai kebutuhan. Pada dasarnya, papan diinstansiasi dengan ukuran nol. Lalu, objek Board dimasukkan petak yang terdapat pada konfigurasi. Pada saat memasukkan petak itulah daftar petak di-*resize* dengan ukuran sesuai dengan indeks petak terbesar. Jadi, saat proses memasukkan petak, program mengecek terlebih dahulu apakah indeks tersebut lebih dari ukuran daftar petak saat ini. Jika lebih besar, daftar petak di-*resize*, jika tidak, daftar petak diabaikan. Petak lalu dimasukkan ke daftar sesuai dengan indeksnya. Dengan ini, papan menjadi dinamis sesuai dengan konfigurasinya.

5.2. Bonus Kreasi Mandiri

Kreasi mandiri pada pengerjaan tugas besar ini dititikberatkan pada kreasi *Graphical User Interface (GUI)* yang dibuat. *GUI* dikreasikan dengan menampilkan grafik papan isometrik selama gim berjalan. Dengan hal tersebut, pemain menjadi tahu bagaimana kondisi papan tanpa harus melakukan perintah CETAK_PAPAN. Pemain tinggal melihat saja grafik papan yang ada. Grafik papan tersebut juga menampilkan informasi-informasi tambahan, seperti kode petak, warna petak, dan posisi pemain. Hal ini menjadikan *user experience* yang didapat para pemain menjadi lebih baik, karena pemain tidak perlu memenuhi *console* dengan perintah-perintah terkait informasi papan.

GUI yang dibuat juga ditampilkan kartu informasi pemain yang sedang melakukan gilirannya yang berisi nama, saldo, dan status pemain. Program juga menampilkan akta suatu petak dalam bentuk kartu setelah pemain melempar dadu. Kedua hal ini juga memberikan *user experience* yang lebih tinggi dibandingkan hanya menyediakan informasi tersebut dari perintah di dalam *console* saja.

6. Pembagian Tugas

Modul (dalam poin spek)	Implementer	Tester
Papan Permainan / Board	13524048	13524048
Petak / Tile	13524048, 13524013	13524048, 13524013
Pemain / Player	13524003, 13524050	13524003, 13524050
Dadu / Dice	13524050	13524050, 13524084
Kartu / Card	13524050	13524050, 13524084
Deck Kartu / Deck	13524050	13524050, 13524084
Manajemen Game / GameManager	13524050, 13524013	13524013
Pemrosesan Perintah / CommandHandler	13524050, 13524013	13524048, 13524013
Lelang / AuctionManager	13524050	13524050

Save & Load Data / DataManager	13524013, 13524003	13524013, 13524003
Transaction Logger / Logger	13524048	13524048, 13524050
Manajemen Validitas / GameException	13524048, 13524013	13524048, 13524013
GUI Utama / NimonspoliGUI	13524084	13524084
Konfigurasi Render / RenderConfig	13524084	13524084
Renderer Papan / BoardRenderer	13524084, 13524050	13524084
Renderer Petak / TileRenderer	13524084	13524084
Menampilkan Pop up / TilePopUpCall / TilePopup / PopUpField / PopUpData	13524084	13524084
CurrentPlayerStats	13524084	13524084
Renderer Bangunan / BuildingRenderer	13524084	13524084
Transformasi Isometrik / IsoTransformer	13524084	13524084
Renderer Bidak / PawnRenderer	13524084	13524084
Konsol Game / GameConsole	13524084, 13524050	13524084

Lampiran

Lampiran A. Form Asistensi

Form asistensi kelompok dapat diakses pada tautan berikut.

📎 [Kumpulan Laporan Asistensi](#)